

The `alethe-dev` branch of `lean-smt`: a summary of the changes by code structure

September 2026

Abstract

The branch `alethe-dev` adds an Alethe proof checker to `lean-smt`, in 130 commits over the merge base `76b4eea` of `main`. This note describes what was added, organized by code structure rather than by chronology: the parser, the changes to the pre-existing library, the new verified modules placed inside that library, and the reconstruction of the Alethe rules. For each part it gives its size, and for the rule reconstruction it lists, rule by rule, which existing `lean-smt` theorem, tactic, verified program or meta-level reconstructor is leveraged.

1 Overview

The additions fall into five parts. Three are the ones one expects, namely a parser for Alethe, the reconstruction of Alethe rules into Lean terms, and changes to routines that `lean-smt` already had. Two more are worth separating out: a set of new *verified* modules that were placed inside the existing reconstruction tree because both the `cvc5` path and the Alethe path use them, and the driver that checks a proof step by step in the kernel, which has no counterpart in the tactic-based `cvc5` path. Figure 1 shows how a proof flows through these parts, and Table 1 gives their sizes. The pre-existing lemma libraries, that is every `Lemmas.lean`, every `Rewrites.lean` and `Prop/Core.lean` under `Smt/Reconstruct/`, are byte-identical to `main`.

2 The parser

The parser is self-contained and depends on nothing in the reconstruction library beyond the S-expression reader `Smt/Data/Sexp.lean`, which existed on `main`.

`Syntax.lean` (205 lines)

The abstract syntax of Alethe proofs, parametric in the representation τ of terms: step arguments `Arg`, `StepData` (id, rule, clause, premises, arguments), `Command` (assume, step, anchor, subproof) and `Proof`. It also defines the hash-consed `Arena` of S-expression `Nodes` over `Nat` ids, with the queries the realizer needs (whether a node is open under a binder, its size, its binder variables).

`Parser.lean` (508 lines)

Turns the S-expressions of a proof into a `Proof Nat` over the arena. Every binding construct is eliminated here through one environment: parallel `let`, `:named` (whose atoms are computed lazily and once, so that sharing survives), user-level `define-fun`, and the variable renamings and assignments of `anchor`. Binders close their variables, so a closed named term is resolved once for the whole proof.

`Realize.lean` (233 lines)

Turns the `Proof Nat` into a `Proof cvc5.Term`. The SMT-LIB problem and every term of the proof are parsed by `cvc5`'s own parser, through `lean-cvc5`'s `InputParser`. Shared closed subterms of the arena are sent once, bound with `:named`, and referred to by name

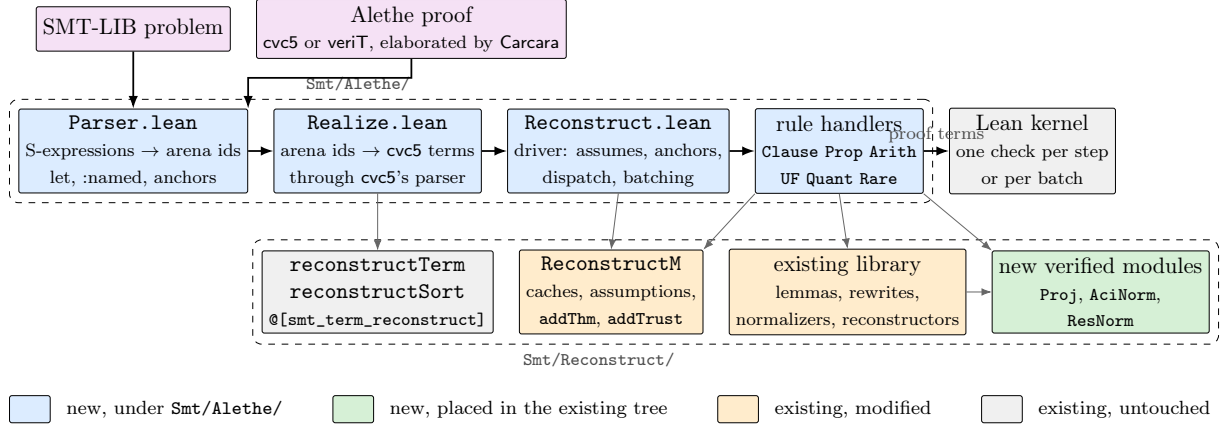


Figure 1: A proof through the checker. The upper row is new; the lower row is the pre-existing reconstruction library, which the checker reaches through `cvc5`'s term representation. The realizer hands every term to `cvc5`'s own parser precisely so that the term reconstructors apply unchanged.

afterwards; open subterms are printed inline. The point of this design is that the reconstructor never sees an Alethe-specific term: `Step.lits` and `Premise.lits` are arrays of `cvc5.Term`, so the whole `@[smt_term_reconstruct]` and `@[smt_sort_reconstruct]` machinery of `lean-smt` applies unchanged.

3 Changes to the pre-existing library

Eleven files that existed on `main` under `Smt/Reconstruct/` were modified, for a net of +1361 / -791 lines. Not all of it is optimization. Each change is classified below as a performance optimization of an existing `cvc5` routine (P), a generalization so that the Alethe checker can call an existing routine (G), a bug fix found through the Alethe evaluation (F), or a new definition (N). Figure 2 shows where the changes land; the classes are detailed in the following subsections and summarized in Table 2.

3.1 Performance optimizations (P)

These change the proof terms the `cvc5` path produces, never its conclusions; each keeps the previous construction as a fallback where one is needed.

- **The polynomial normalizer sorts once.** In `Int/Polynorm.lean`, `Rat/Polynorm.lean` and `Real/Polynorm.lean`, `Expr.toPolynomial` used to fold a sum through a sorted insertion, so that a sum of n terms cost the kernel $O(n^3)$ steps. It now collects the monomials in one pass (`Expr.flatten`, threading a sign for `Int` and a rational multiplier for `Rat` and `Real` so that negation, subtraction and division by a constant cost nothing) and sorts them with a bottom-up merge sort (`mergePairs`, `mergeAll`, `normalize`), all structurally recursive because the kernel does not unfold well-founded recursion. The definitions and their denotation lemmas were rewritten and reproved; the exported soundness theorem `denote_eq_from_toPolynomial_eq` is unchanged in statement and proof, and nothing was sorried. Commit 7258b45. Measured on a 400-variable sum: 25s and 8.9 GB became 1.3s.
- **polyNorm decides the equality.** The kernel tactic closed its goal with `Eq.refl` between two normalizer applications, which the kernel unifies by unfolding both sides in lockstep instead of evaluating each; on large coefficients that ran for over half an hour. It now applies `of_decide_eq_true` to `Eq.refl true`, as `nativePolyNorm` already did. Meta code only; `nativePolyNorm` itself is untouched. Commit 711e8e7.
- **The monomial order is keyed by Nat.** In `Int/Polynorm.lean` and `Rat/Polynorm.lean`,

Part	Files	Lines
Alethe parser	Syntax, Parser, Realize (3 new)	952
Rule reconstruction	Basic, Clause, Prop, Arith, UF, Quant, Polyeq, Lemmas, Rare, Rare/Basic (10 new)	3 120
Generated RARE table	Rare/Rules.lean (1 new; generator gen.py 445, input rewrites.eo 513)	1 379
Driver, frontend, tactic	Reconstruct, Frontend, Command, Tactic, Alethe.lean (5 new)	1 381
New verified modules in the existing tree	Prop/AciNorm 461, Prop/Proj 239, Prop/ResNorm 397 (3 new)	1 097
Changes to pre-existing library files	11 modified	+1 493 / -814
Registration	Smt.lean, Attribute.lean, Options.lean (3 modified)	+8
Tests	Test/Alethe: 49 test files over 96 problem and proof files	266 + 15 061
Unit tests	Test/Unit/AciNorm, Test/Unit/ResNorm (2 new)	126
Scripts	check-alethe.sh, setup-alethe.sh, rule-boxplots.py (3 new)	657
Report and notes	docs-alethe/: $\text{\TeX}$ 2 478, Markdown 606, plus figures and CSVs	

Table 1: Size of each part of the branch. Lines are lines of Lean unless stated otherwise. The 130 commits are on top of 76b4eea.

`Polynomial.addAux` asked `m.vars < n.vars` and then `m.vars = n.vars`: two traversals of the same two lists, each through a `Decidable` instance chain, which in the rational normalizer put a `Bool <` under an `Or` and an `And`. Since the kernel is what evaluates this — on a profiled `la_generic` step, 93% of the kernel’s time was the one `decide` the normalizer hands it, and 83% of that was this merge — those chains are what it spent its time unfolding. New `cmpVar`, `bleVar` and `cmpVars` decide the order with `Nat.bl`, `Nat.beq` and `Nat.ble`, which the kernel evaluates as primitives, and compare two monomials’ variables three ways in one pass. The order itself is unchanged, so nothing downstream moves. `Monomial.add` drops its `māĈĀ.vars = māĈĈ.vars` argument, which only guarded the call: `addAux` can then match on `cmpVars` without a dependent motive, and `denote_addAux` recovers the equation from the new `cmpVars_eq`. The denotation lemmas were reproved; nothing was sorried. Commit 6dec36a. A/B in one process, the old merge kept beside the new one on the same monomials: 2.2× at 176 monomials. Over the corpus it is 1.12× on `la_generic` for both solvers and 1.06× on `poly_simp`; the `native` arm, which evaluates the same normalizer as compiled code, moved by 1%, which is the control that says the attribution is right.

- **Projection instead of list indexing.** In `Prop.lean`, the `cvc5` rules `AND_ELIM`, `NOT_OR_ELIM`, `CNF_AND_POS`, `CNF_AND_NEG`, `CNF_OR_POS` and `CNF_OR_NEG` projected the *i*-th conjunct through `andN` and `List.getElem` with a decided bound, charging every step for the whole conjunction. They now walk the right-nested `And` the hypothesis already is, through the new `Prop/Proj.lean` (Section 4). `CNF_OR_POS` collapses to the single node `notOrSelf`. Commit 008ac35.
- **ACI_NORM through the reflective normalizer.** In `Builtin.lean`, an `and/or` layer is normalized by the verified `AciNorm` normalizer, evaluated by the kernel; other operators and any failure fall back to the previous AC rewriting tactic. Kernel time for a 200-atom layer drops from seconds to about 30 ms. Commit 28bb705.
- **Decidable instances and closed-term caching.** In `Smt/Reconstruct.lean`,

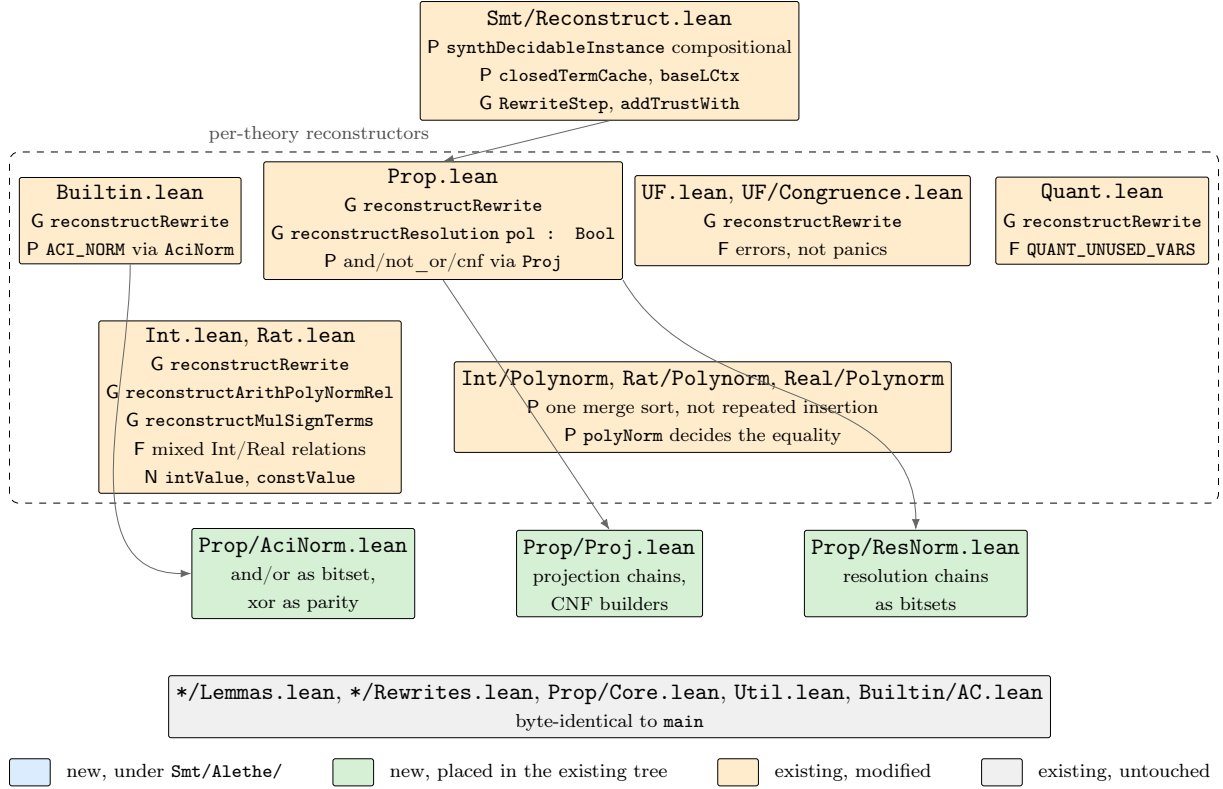


Figure 2: Where the branch touches the existing reconstruction tree. Every per-theory file received the same generalization of its rewrite reconstructor; the optimizations concentrate in the polynomial normalizers and in `Prop.lean`, the latter through the new `Proj` module. The lemma libraries were not modified.

`synthDecidableInstance` guards the instance search (whose own “failed to synthesize” used to escape and take down whole runs on `QF_UFIDL`) and, when the search fails, builds the instance compositionally through $\neg$, $\wedge$, $\vee$ and $\leftrightarrow$ so that it matches the instance a theorem over `[Decidable c]` elaborates to. The reconstruction state gains a `closedTermCache`, valid across binder scopes, enabled only when a driver sets `baseLCtx`; the `cvc5` path leaves it off.

3.2 Generalizations for sharing (G)

Pure refactors with no behavior change on the `cvc5` path, almost all in the first commit `c69e4d5`.

- A record `RewriteStep` (rule, arguments, list arguments, result, lazily reconstructed premises) in `Smt/Reconstruct.lean` decouples a rewrite step from `cvc5.Proof`. Every per-theory `reconstructRewrite` in `Builtin`, `Int`, `Prop`, `Quant`, `Rat` and `UF` was retyped to take it, and each `cvc5` call site wraps its proof with `RewriteStep.ofProof`. This is what lets an Alethe `rare_rewrite` or `qnt_join` step, which has no proof object, reach the existing rewrite reconstructors.
- `Prop.getResolutionResult` and `Prop.reconstructResolution` take the pivot polarity as a `Bool` rather than a `cvc5` term; `Prop.isNotOf` became public.
- In `Int.lean` and `Rat.lean`, `reconstructArithPolyNormRel` takes the premise’s term and proof instead of a proof object, and the term-level half of the mult-sign reconstruction was factored out as `reconstructMulSignTerms`, the old name staying as a two-line wrapper (commit `eab472d`).
- `addTrustWith` lets a step be trusted with a custom trace message.

File	+	−	Classes
Smt/Reconstruct.lean	97	10	P G
Builtin.lean	50	40	G P
Int.lean	114	97	G F N
Int/Polynorm.lean	268	98	P
Prop.lean	134	142	G P
Quant.lean	67	54	G F
Rat.lean	121	86	G F N
Rat/Polynorm.lean	320	137	P
Real/Polynorm.lean	263	121	P
UF.lean	21	21	G
UF/Congruence.lean	16	8	F

Table 2: The modified pre-existing files under `Smt/Reconstruct/` and the classes of change each received.

3.3 Bug fixes (F)

- `Quant.lean`: `QUANT_UNUSED_VARS` was wrong when a prefix binds the same variable twice or reorders (`cvc5` reuses the variable node for a name, and the body refers to the innermost binder), and again when every binder is dropped and the body is itself a quantifier. Each side is now instantiated from the binders of the other. Commits `3ca2f21`, `4d88b28`. A regression test with a repeated binder was added.
- `Int.lean`, `Rat.lean`: a relation with one `Int` side and one `Real` side was claimed by the `Int` handler and reconstructed ill-sorted. The `Int` handler now declines unless both sides are integer, and the `Rat` handler casts through a new `ratArg` helper.
- `UF/Congruence.lean`: the left- and right-associative congruence helpers replaced panicking `Option.get!` and `hs[0]!` with errors, so a premise that is not an equality is reported instead of becoming a dummy term the kernel then rejects. Commit `ad066ae`.

3.4 New definitions (N)

`Int.intValue` and `Rat.constValue` read a coefficient written as Alethe writes it, as a negated literal (`- 1`) or a quotient of literals (`/ 1 4`); for `cvc5` input they agree with the previous direct reads. No new theorem was added to any pre-existing file.

4 New verified modules in the existing tree

Three modules were added under `Smt/Reconstruct/Prop/` rather than under `Smt/Alethe/`, because they are reflective checkers in the style of the polynomial normalizer and serve both paths. Each has a verified core of ordinary definitions and theorems, and a small meta layer that reifies an expression and applies the soundness theorem as a term.

`Prop/Proj.lean` (239 lines, 8 theorems)

Projection out of a right-nested $\wedge$ or $\neg\vee$ chain, and the builders of the CNF axioms (`mkAndProj`, `mkNotOrProj`, `mkOrInj`, `mkCnfAndPos`, `mkCnfAndNeg`, `mkCnfOrNeg`, `notOrSelf`). A `ProjChain` keeps the partially unrolled spine so that a step projecting many indices out of one hypothesis builds $O(n)$ nodes in total. Used by the `cvc5` rules named above and by the Alethe rules `and`, `not_or`, `and_pos`, `and_neg`, `or_pos`, `or_neg`, `onepoint` and `distinct_elim`.

`Prop/AciNorm.lean` (461 lines, 20 theorems)

One layer of $\wedge$ or $\vee$ is reified into a tree over indexed atoms and normalized to a bitset, `none` when the annihilator occurs; a layer of `xor` is normalized to the parity of its atoms.

The kernel evaluates the normal form with its accelerated `Nat` bit operations. Used by `cvc5`'s `ACI_NORM` and by Alethe's `semilattice_simp` and `boolean_group_simp`.

`Prop/ResNorm.lean` (397 lines, 18 theorems)

A checker for resolution chains and the containment rules. A literal is a `Nat`, a clause is both a `List Nat` whose denotation unfolds definitionally to the stated proposition and a bitset on which each link of a chain is a few accelerated operations; the pivots are hints, so a wrong one fails only at the final containment. It depends on `propext` and `Quot.sound` only. Used by the Alethe clausal rules behind the option `smt.alethe.reflect`, default off in the library and set by the evaluation's runner; the `cvc5` path does not use it yet. It carries `resolution` and the `contraction/reordering` steps that really permute or deduplicate. It does *not* carry `or` and `weakening`, nor a `contraction` whose premise is already its conclusion: those paths do essentially no work already — for `or` the premise's proposition is the stated clause — and reifying two clauses for them was measured 3 to 8 times slower.

5 Reconstruction of the Alethe rules

The rule handlers are registered with a new attribute `alethe_rule_reconstruct`, added to the existing attribute framework in three lines of `Smt/Attribute.lean` and collected by the existing `getReconstructors`. Five handlers cover the rules, one per family (`Clause`, `Prop`, `Arith`, `UF`, `Quant`), with `Rare` dispatched from `UF`.

This section covers the rules that reach the checker after Carcara's pipeline as the evaluation runs it: `polyeq`, `local`, `core-simp-rare` restricted by `-core-rules` to the `*_simplify` family and the legacy `ac_simp`, `aci_simp` and `absorb`, `budget`, and the `reordering` pass. Those passes remove the `*_simplify` rules and the legacy ACI rules (replaced by `rare_rewrite` and `evaluate` chains and by the structural `semilattice_simp`, `boolean_group_simp` and `assoc_simp`) and `reordering`. The non-linear multiplication rules are left out as well. The handlers also accept those rules, so that a raw solver proof can be checked, but they are not covered here. The reconstructors of `bfun_elim`, `th_resolution` and `strict_resolution` have been dropped, no corpus of the evaluation emitting them after the pipeline. Figure 3 shows which existing component each family draws on; the subsections list every rule. Throughout, a component is *existing* if it is present on `main` and *new* otherwise; the new verified modules of Section 4 are marked as such.

5.1 Substrate shared by every rule

Before any rule, the checker reuses the reconstruction monad and its services unchanged: `ReconstructM` with `Reconstruct.Context` and `Reconstruct.State`; the term and proof caches, `withNewTermCache` and `withNewProofCache`; the assumption handling, `withAssums` and `findAssumWithType?`; the step recorders `addThm`, `addTac` and `addTrust`, and the option `useNative`. Terms and sorts go through `reconstructTerm`, `reconstructSort` and `reconstructSortLevelAndSort`, hence through every `@[smt_term_reconstruct]` and `@[smt_sort_reconstruct]` handler of `Prop`, `Builtin`, `UF`, `Int`, `Rat`, `Real`, `BitVec` and `Datatype`. The alethe tactic reuses the entire `smt` front end: `Preprocess.mono`, `pushHintsToCtx`, `intros`, `negateGoal`, `normalize`, `embedding`, `applySteps`, `genUniqueFVarNames`, `prepareSmtQuery`, `Command.cmdsAsQuery` and the `Config/Result` types. The problem's symbols are introduced with `reconstructSort` through a flat `withLocalDecls` rather than one nested `withLocalDecl` per declaration, and choice symbols are bound to `Classical.epsilon` over `reconstructSortLevelAndSort`.

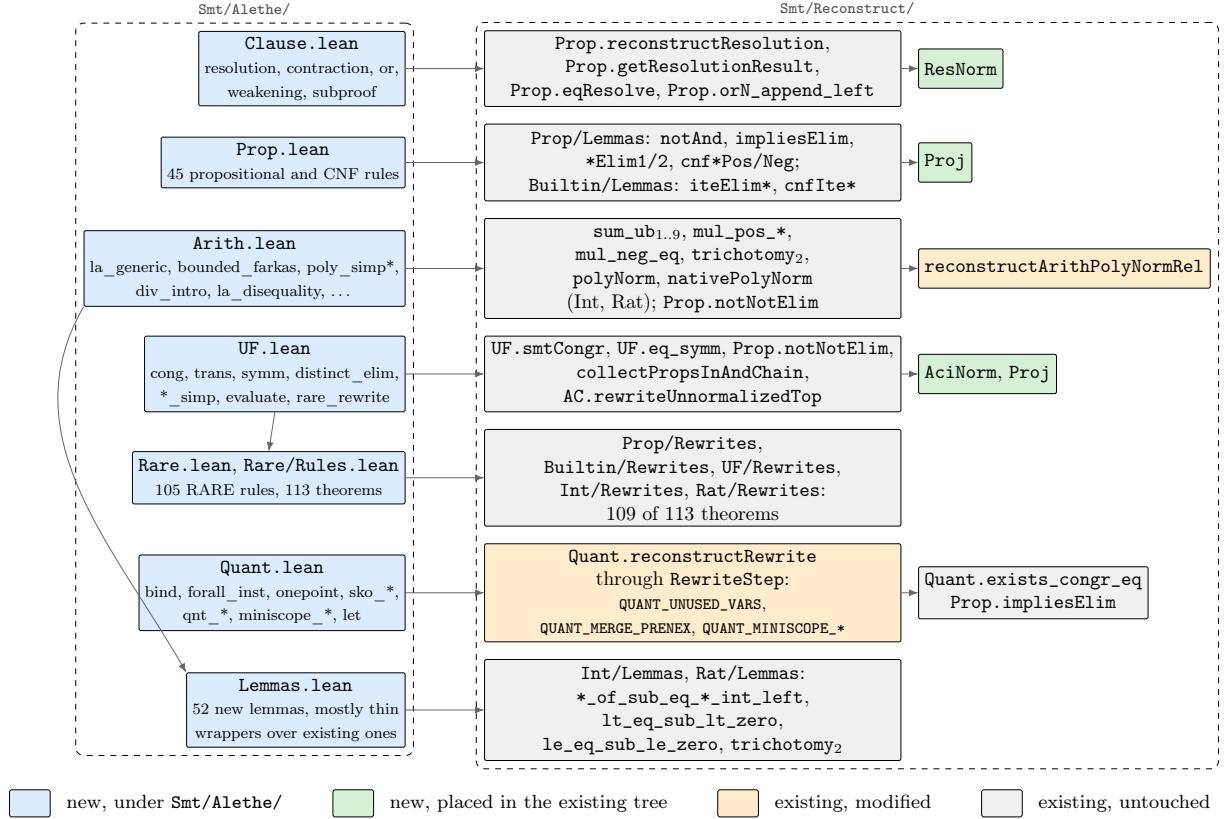


Figure 3: What each rule family leverages. Grey boxes are components that existed on `main` and were not modified; orange ones existed and were generalized or factored out on the branch; green ones are the new verified modules of Section 4.

5.2 Clausal rules (`Clause.lean`, 227 lines)

Rule	Component
<code>resolution</code>	existing <code>Prop.reconstructResolution</code> , <code>Prop.getResolutionResult</code> ; new <code>ResNorm.proveClause</code> when <code>smt.alethe.reflect</code> is set
<code>contraction</code>	new lemma <code>orN_of_map</code> ; <code>ResNorm</code> when set, unless the premise is already the conclusion
<code>or</code>	new lemma <code>orN_of_map</code>
<code>weakening</code>	existing <code>Prop.orN_append_left</code>
<code>subproof</code>	new lemma <code>orN_of_impliesN</code>
<code>hole</code>	trusted, through <code>addTrustWith</code>

The fix-up that reconciles a computed clause with the stated one (permutations, duplicates, flattening) uses the existing `Prop.eqResolve` and the existing AC tactic `Meta.AC.rewriteUnnormalizedTop` from `Builtin/AC.lean` as its last resort. Resolution on a premise that itself holds the pivot no longer re-resolves. The existing resolution reconstructor is the largest single reuse in the checker.

5.3 Propositional rules (`Prop.lean`, 266 lines)

Of the 45 rules, 40 are direct applications of lemmas that existed in `Prop/Lemmas.lean` and `Builtin/Lemmas.lean`.

Rule	Component
<code>not_and</code>	existing <code>Prop.notAnd</code>
<code>implies</code>	existing <code>Prop.impliesElim</code>
<code>not_implies1, not_implies2</code>	existing <code>Prop.notImplies1, Prop.notImplies2</code>
<code>equiv1, equiv2</code>	existing <code>Prop.equivElim1, Prop.equivElim2</code>
<code>not_equiv1, not_equiv2</code>	existing <code>Prop.notEquivElim1, Prop.notEquivElim2</code>
<code>xor1, xor2</code>	existing <code>Prop.xorElim1, Prop.xorElim2</code>
<code>not_xor1, not_xor2</code>	existing <code>Prop.notXorElim1, Prop.notXorElim2</code>
<code>ite1, ite2</code>	existing <code>Builtin.iteElim2, Builtin.iteElim1</code>
<code>not_ite1, not_ite2</code>	existing <code>Builtin.notIteElim2, Builtin.notIteElim1</code>
<code>implies_pos</code>	existing <code>Prop.cnfImpliesPos</code>
<code>implies_neg1, implies_neg2</code>	existing <code>Prop.cnfImpliesNeg1, Prop.cnfImpliesNeg2</code>
<code>equiv_pos1, equiv_pos2</code>	existing <code>Prop.cnfEquivPos2, Prop.cnfEquivPos1</code>
<code>equiv_neg1, equiv_neg2</code>	existing <code>Prop.cnfEquivNeg2, Prop.cnfEquivNeg1</code>
<code>xor_pos1, xor_pos2</code>	existing <code>Prop.cnfXorPos1, Prop.cnfXorPos2</code>
<code>xor_neg1, xor_neg2</code>	existing <code>Prop.cnfXorNeg2, Prop.cnfXorNeg1</code>
<code>ite_pos1, ite_pos2</code>	existing <code>Builtin.cnfItePos2, Builtin.cnfItePos1</code>
<code>ite_neg1, ite_neg2</code>	existing <code>Builtin.cnfIteNeg2, Builtin.cnfIteNeg1</code>
<code>and, not_or</code>	new <code>Proj.mkAndProj, Proj.mkNotOrProj</code>
<code>and_pos, and_neg, or_neg</code>	new <code>Proj.mkCnfAndPos, mkCnfAndNeg, mkCnfOrNeg</code>
<code>or_pos</code>	new <code>Proj.notOrSelf</code>
<code>true</code>	trivial
<code>false, not_not</code>	new lemmas <code>not_false_cl, not_not_cl</code>
<code>ite_then_intro, ite_else_intro</code>	new lemmas of the same names
<code>connective_def, qnt_duality</code>	new lemmas <code>connective_def_xor/eq/ite</code> ; quantifier cases by a small <code>simp</code> set; existing <code>synthDecidableInstance</code>
<code>and_intro</code>	new meta code, an <code>And.intro</code> fold

5.4 Arithmetic rules (Arith.lean, 553 lines)

The Farkas driver behind `la_generic`, `la_tautology` and `bounded_farkas` is new meta code, about 340 lines: the negations of the clause's literals are assumed, oriented into bounds, strengthened for strict integer bounds, scaled by the coefficients given as `:args`, summed, and refuted through the normalized difference of the two sides and a decidable fact about the resulting constant. Every proof-level ingredient of that driver is existing.

Rule	Component
<code>la_generic, la_tautology, bounded_farkas</code>	existing <code>Int.sum_ub1..9, Rat.sum_ub1..9</code> ; existing <code>mul_pos_lt/le/eq, mul_neg_eq (Int, Rat)</code> ; existing <code>Int.polyNorm, Rat.polyNorm</code> and their native variants; existing <code>Prop.notNotElim</code> ; Lean core <code>omega</code> for integer tightening; new lemmas <code>farkas_lt/le/eq (Int, Rat)</code> , <code>orN_of_impliesN_not</code>
<code>poly_simp</code>	existing <code>Int.polyNorm, Rat.polyNorm</code> , and native variants
<code>poly_simp_rel</code>	<code>reconstructArithPolyNormRel (Int, Rat)</code> , generalized from the existing handler; nine new mixed-sort lemmas over the existing <code>Rat.*_of_sub_eq*_int_left</code>
<code>la_rw_eq</code>	new lemmas <code>Int.la_rw_eq, Rat.la_rw_eq</code> over existing <code>Rat.trichotomy2</code>
<code>la_disequality</code>	new lemmas over existing <code>Rat.trichotomy2</code>
<code>la_totality</code>	new lemmas <code>Int.la_totality, Rat.la_totality</code>
<code>div_intro</code>	new lemmas <code>div_intro_pos/neg/real</code> ; <code>omega</code> for a constant divisor
<code>div_by_zero_intro</code>	new lemma <code>div_by_zero_intro</code>
<code>lia_generic</code>	Lean core <code>omega</code> only; a step outside its fragment is trusted

The new Farkas, disequality, totality and `la_rw_eq` lemmas in `Alethe/Lemmas.lean` are themselves proved from existing `Rat.trichotomy2`, `Rat.lt_eq_sub_lt_zero`, `Rat.le_eq_sub_le_zero`, `Rat.lt_of_le_of_lt`, `Rat.lt_of_lt_of_le` and `Rat.le_of_lt`. No lean-smt tactic is invoked by name: the polynomial normalizer is called as a function on a goal, and `omega` is Lean's.

5.5 Equality and rewriting rules (UF.lean, 516 lines)

`distinct_elim` is the one rule whose handler got *smaller* rather than larger. A `distinct` over n terms eliminates to the conjunction of its $\binom{n}{2}$ disequalities, and `veriT` writes those conjuncts in any order and either orientation, so the handler used to align the stated conjunction against the one it builds and bridge the two with a congruence chain. That alignment cost 5.17 CPU hours over the corpus, 1.58 seconds a step, and — as the `native` arm showed, by paying the same 5.13 hours for steps whose congruences it never reduces — the cost was the alignment itself, not the kernel. Carcara’s `polyeq` pass now emits the canonical elimination, so the handler compares the conjuncts in order and closes the step with an `Eq.refl` ascribed to the stated equality, refusing anything else. The rule falls to 0.04 CPU hours and no step in the corpus became trusted.

Rule	Component
<code>cong</code>	existing <code>UF.smtCongr</code> , <code>UF.eq_symm</code> ; new <code>transportEq</code> for <code>distinct</code>
<code>eq_congruent</code> , <code>eq_congruent_pred</code>	existing <code>UF.smtCongr</code> , <code>Prop.notNotElim</code> ; new lemma <code>orN_of_impliesN_not</code>
<code>eq_transitive</code>	existing <code>Prop.notNotElim</code> ; new chaining over <code>Eq.trans</code>
<code>eq_symmetric</code>	existing <code>UF.eq_symm</code> , <code>Prop.impliesElim</code>
<code>refl</code> , <code>eq_reflexive</code>	existing <code>reconstructSortLevelAndSort</code>
<code>symm</code> , <code>not_symm</code> , <code>trans</code>	new meta code over <code>Eq.symm</code> , <code>Ne.symm</code> , <code>Eq.trans</code> , with defeq-aware orientation under anchor <code>lets</code>
<code>distinct_elim</code>	existing <code>collectPropsInAndChain</code> ; new lemma <code>distinct_bool_false</code> ; otherwise an <code>Eq.refl</code> ascribed to the stated equality, since Carcara canonicalizes the elimination
<code>semilattice_simp</code>	new <code>AciNorm.proveEq</code>
<code>boolean_group_simp</code>	new <code>AciNorm.proveXorEq</code>
<code>assoc_simp</code>	existing <code>Meta.AC.rewriteUnnormalizedTop</code>
<code>evaluate</code>	existing <code>synthDecidableInstance</code> ; new <code>decideProofOfNative</code> , <code>proveByCases</code> , <code>decideGround</code>
<code>rare_rewrite</code>	dispatched to <code>Rare.lean</code> , below

5.6 RARE rewrites (Rare.lean 109, Rare/Basic.lean 44, Rare/Rules.lean 1 380 lines)

A `rare_rewrite` step names a rule of the RARE file Carcara was run with; the `core-simp-rare` pass replays every `*_simplify` step as a chain of such steps and `evaluate`. The generator `gen.py` reads that file, `rewrites.eo`, and emits one theorem per rule, two for rules polymorphic over `Int` and `Real`, plus a dispatch table describing each rule’s arguments; existing proofs are preserved across regenerations. `Rare.lean` instantiates the theorem with the step’s arguments by a reducing telescope and unification.

Leaving aside the two non-linear `mult-tangent` rules, the table holds 105 rules and 113 theorems. Their proofs are almost entirely existing lemmas:

Proof of the generated theorem	Theorems
existing <code>Int.Rewrite.*</code> and <code>Rat.Rewrite.*</code> lemmas (<code>Int/Rewrites</code> , <code>Rat/Rewrites</code>)	45
existing <code>Prop.bool_*</code> and <code>Prop.ite_*</code> lemmas (<code>Prop/Rewrites</code>)	41
existing <code>Builtin.ite_*</code> and <code>UF.eq_*</code> , <code>UF.distinct_binary_elim</code> (<code>Builtin/Rewrites</code> , <code>UF/Rewrites</code>)	13
<code>rfl</code>	8
Lean core <code>Int.emod_emod</code> , <code>Int.emod_def</code> , or an inline <code>propext</code>	3
short tactic proof	2
new lemma <code>or_not_refl</code> (a rule newer than <code>lean-cvc5</code> ’s enumeration)	1

Six `array-*` rules have no statement, there being no array sort support, and are trusted. The rule `distinct-false` is special-cased in `UF.lean` through `Proj.mkAndProj`.

5.7 Quantifier rules (Quant.lean, 655 lines)

Rule	Component
<code>qnt_rm_unused</code> ($\forall$)	existing <code>Quant.reconstructRewrite</code> at <code>QUANT_UNUSED_VARS</code>
<code>qnt_join</code>	existing <code>Quant.reconstructRewrite</code> at <code>QUANT_MERGE_PRENEX</code>
<code>miniscope_distribute</code>	existing <code>Quant.reconstructRewrite</code> at <code>QUANT_MINISCOPE_AND</code>
<code>miniscope_split</code>	existing <code>Quant.reconstructRewrite</code> at <code>QUANT_MINISCOPE_OR</code>
<code>miniscope_ite</code>	existing <code>Quant.reconstructRewrite</code> at <code>QUANT_MINISCOPE_ITE</code>
<code>forall_inst</code>	existing <code>Prop.impliesElim</code> ; new <code>alignInstances</code>
<code>bind</code>	existing <code>Quant.exists_congr_eq</code> , Lean core <code>forall_congr</code> ; new lemma <code>orN_forall</code>
<code>onepoint</code>	new <code>Proj.mkOrInj</code> , <code>Proj.mkAndProj</code> ; existing <code>collectPropsInOrChain</code> , <code>collectPropsInAndChain</code> ; new lemmas <code>onepoint_forall</code> , <code>onepoint_exists</code>
<code>sko_ex</code> , <code>sko_forall</code>	new lemmas <code>sko_ex_eq</code> , <code>sko_forall_eq</code>
<code>qnt_simplify</code>	new lemmas <code>qnt_forall_true/false</code> , <code>qnt_exists_true/false</code>
<code>let</code>	new meta code, a <code>mkCongrArg</code> chain over anchor <code>lets</code>
<code>qnt_rm_unused</code> ($\exists$)	new meta code; the shared reconstruction is $\forall$ -only

The first five rows are handled with no new proof code at all: the Alethe rule name is mapped to the `cvc5` rewrite rule and the existing reconstructor is called through a `RewriteStep`. The anchor machinery (kept variables introduced first, assignments `let`-bound so that the substitution is definitional) is new and lives in the driver.

5.8 Assumptions (Polyeq.lean, 145 lines)

An `assume` must be an assertion of the problem. Carcara’s `polyeq` pass makes that syntactic on its side: an `assume` matching an assertion only up to a flipped equality, a regrouped associative application or renamed bound variables is replaced by the assertion, with explicit steps deriving the proof’s spelling. One difference survives the pass because Carcara’s parser erases it: numerals are identified by value, so `0.0` and `0/1` are one constant to Carcara, the `assume` is an exact match there, nothing is elaborated, and the printer chooses its own spelling. The checker realizes both sides through `cvc5`’s parser, which is syntactic, and sees two terms. `Polyeq.lean` accepts exactly that: `polyeq` compares two `cvc5` terms structurally with numerals by value, and `polyeqProof?` proves the propositions equal by congruence down to the numerals, where the kernel unfolds one spelling into the other. A flipped equality or a renamed binder is rejected, as `Test/Alethe/M0/assume_flipped` checks. The path runs only after the exact and definitional matches fail; on the sample run in the repository no assumption reached it.

5.9 The new lemmas

`Alethe/Lemmas.lean` holds 52 theorems. They cover the clausal shapes of Alethe conclusions (`orN_of_impliesN`, `orN_of_impliesN_not`, `orN_of_map`, `orN_of_getElem`, `orN_forall`), the Farkas, disequality, totality and `la_rw_eq` statements in `Int` and `Rat`, nine mixed-sort statements for `poly_simp_rel`, the Skolemization and one-point equalities, the four `qnt_simplify` constants, the three `connective_def` shapes, the `div_intro` family, and small facts about `distinct`. None duplicates a library lemma; most are thin wrappers over the existing `Int` and `Rat` lemmas named above.

5.10 What has no counterpart in the library

Genuinely Alethe-specific new work, with no `lean-smt` component to lean on: the Farkas driver of `la_generic`; the anchor and binder rules `bind`, `let`, `onepoint`, `sko_ex`, `sko_forall`; the numeral matching of assumptions; the term-orientation logic of `trans`, `symm` and `cong` under anchor `lets`; and the step-by-step kernel driver of `Reconstruct.lean`, 775 lines, with its batching (`smt.alethe.batch`, `smt.alethe.batchSize`), concurrency (`smt.alethe.jobs`), inner re-check of a rejected batch, closed-term caching across scopes, and CSV timing output.

6 Tests, scripts and documents

`Test/Alethe/` adds 49 tests over 96 problem and proof files, grouped as `MO`, `QF_UF`, `UF`, `QF_LIA`, `QF_LRA`, `QF_UFLIA` and `Tactic`; each test is a single `##check_alethe` command over a problem and its proof, so the Lean side is 266 lines and the proof text 15 061. `Test/Unit/AciNorm.lean` and `Test/Unit/ResNorm.lean` exercise the new normalizers directly; `Test/Unit/Normalize.expected` was refreshed because the `polyNorm` change moves an internal counter.

`scripts/setup-alethe.sh` builds `cvc5`, `Carcara` and the no-Mathlib `lean-smt`; `scripts/check-alethe.sh` runs the checker over a proof with a heap cap, phase timings and optional per-step CSV; `rule-boxplots.py` draws the per-rule plots. `docs-alethe/` holds the evaluation report `alethe-lean-smt.tex` with its figures and the two notes `rule-audit.md` and `elaboration-opportunities.md`.